

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

Наследование классов. Преобразование типов

по курсу: Основы программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № _____ 4321

подпись, дата

Г.В.Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 Цель работы	3
2 Задание	3
3 Описание работы программы.....	4
4 Исходный код программы.....	7
5 Результаты работы программы	18
6 Вывод	19

1 Цель работы

Изучить основы наследования на примере создания иерархий классов. Ознакомиться с механизмами преобразования типов. На основе изученного материала разработать класс с несколькими конструкторами и продемонстрировать его функциональность через наследование и преобразование типов.

2 Задание

Необходимо разработать приложение, моделирующее работу автомобиля, где его компоненты, такие как двигатель и колеса, имеют собственные состояния и могут ломаться. Требуется реализовать базовый абстрактный класс, определить иерархию наследников, а также продемонстрировать работу нескольких конструкторов. Для наглядности приложения разработан интерфейс Windows Forms, который позволяет пользователю тестировать функционал компонентов, создавая объекты, изменяя их состояние и наблюдая за событиями поломок.

3 Описание работы программы

В основе разработки лежит базовый класс `Component`, который представляет общие свойства и поведение всех компонентов автомобиля, таких как состояние (новое или сломанное) и механизм генерации событий при изменении состояния.

```
using System;

// Ссылка: 1
public abstract class Component
{
    // Абстрактный класс Component, который реализует общие методы и события для всех компонентов автомобиля
    // Свойство Condition хранит текущее состояние компонента, по умолчанию "Новое"
    // Ссылка: 7
    public string Condition { get; protected set; } = "Новое";

    // Метод ChangeCondition для изменения состояния компонента
    // Проверяет, что состояние может быть только "Новое" или "Сломано"
    // При изменении состояния на "Сломано" вызывает событие поломки компонента
    // Ссылка: 0
    public void ChangeCondition(string condition)
    {
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        Condition = condition;

        // Если состояние компонента "Сломано", генерируем событие поломки
        if (Condition == "Сломано")
        {
            OnComponentBroken();
        }
    }

    // Защитный метод для вызова события поломки компонента
    // Срабатывает, если компонент сломался
    // Ссылка: 1
    protected virtual void OnComponentBroken()
    {
        ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события ComponentBroken
    }

    // Событие поломки компонента, которое оповещает об изменении состояния на "Сломано"
    public event EventHandler ComponentBroken;
}
```

Рисунок 1 – Файл класса `Component.cs`

Класс `Engine` унаследован от `Component` и реализует функциональность управления двигателем, включая его запуск, остановку и обработку поломок.

```
public class Engine : Component
{
    // Свойство HorsePower хранит мощность двигателя в лошадиных силах
    // Ссылка: 2
    public int HorsePower { get; set; }

    // Свойство IsRunning указывает, работает ли двигатель (true - работает, false - не работает)
    // Ссылка: 3
    public bool IsRunning { get; private set; }

    // Метод Start запускает двигатель, если его состояние не "Сломано"
    // Если двигатель сломан, выбрасывается исключение
    // Ссылка: 1
    public void Start()
    {
        if (Condition == "Сломано")
            throw new InvalidOperationException("Двигатель сломан, его нельзя запустить.");
        IsRunning = true;
    }

    // Метод Stop останавливает двигатель, если его состояние не "Сломано"
    // Если двигатель сломан, выбрасывается исключение
    // Ссылка: 1
    public void Stop()
    {
        if (Condition == "Сломано")
            throw new InvalidOperationException("Двигатель сломан, его нельзя остановить.");
        IsRunning = false;
    }

    // Переопределение метода ChangeCondition для изменения состояния двигателя
    // Изменяет состояние и вызывает событие поломки, если двигатель стал "Сломан"
    // Ссылка: 1
    public void ChangeCondition(string condition)
    {
        Condition = condition;
    }
}
```

Рисунок 2 – Файл класса `Engine.cs`

Класс Wheel аналогично наследует базовый функционал, но представляет поведение колес автомобиля. Класс Car объединяет все компоненты в единую модель, предоставляя их взаимодействие и управление. В нем реализовано несколько конструкторов, которые позволяют инициализировать автомобиль с разными параметрами.

```
public class Wheel : IComponent
{
    Ссылка: 9
    public string Condition { get; set; } = "Новое"; // Состояние колеса
    Ссылка: 2
    public int BrokenCount { get; set; } = 0; // Счетчик поломок колеса

    // Событие поломки колеса
    public event EventHandler ComponentBroken;

    // Метод для восстановления колеса
    Ссылка: 0
    public void Repair()
    {
        // Если колесо уже новое, невозможно выполнить восстановление
        if (Condition == "Новое")
        {
            throw new InvalidOperationException("Колесо уже в хорошем состоянии.");
        }
        // Восстановление состояния колеса в "Новое"
        Condition = "Новое";
        // Генерация события о восстановлении колеса
        ComponentBroken?.Invoke(this, EventArgs.Empty);
    }

    // Метод для изменения состояния колеса
    Ссылка: 3
    public void ChangeCondition(string condition)
    {
        // Проверка, что состояние колеса либо "Новое", либо "Сломано"
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        // Изменение состояния колеса
        Condition = condition;

        // Если колесо сломано, увеличиваем счетчик поломок и генерируем событие
        if (Condition == "Сломано")
        {
            BrokenCount++; // Увеличиваем счетчик поломок
            ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события при поломке
        }
    }
}
```

Рисунок 3 – Файл класса Wheel.cs

На рисунке 4 изображена диаграмма классов программы.

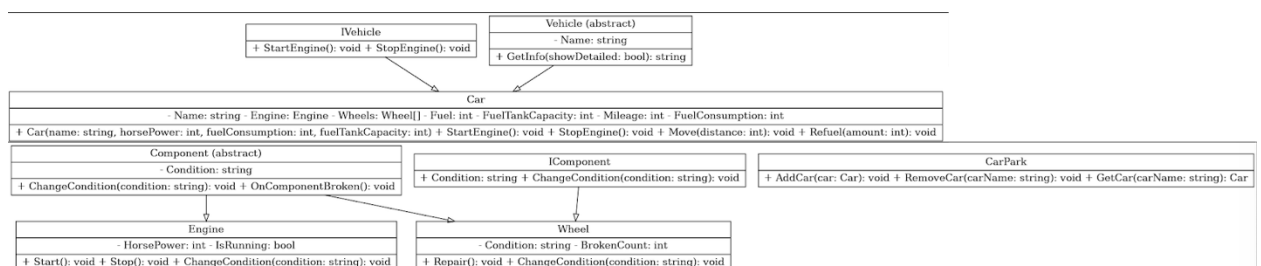


Рисунок 4 – Диаграмма классов

На рисунке 5 представлена структура оформления базового окна Windows Forms

The screenshot shows a Windows Forms application window titled "Car Management". The window has a standard Windows title bar with minimize, maximize, and close buttons. The main area contains several input fields and buttons. On the left, there are four labels: "Введите имя авто:", "Введите мощность авто:", "Введите расход авто:", and "Введите бак авто:". Each label is followed by a text input field. Below these fields are two buttons: "Создать авто" and "Удалить авто". At the bottom left, there is a checkbox labeled "Подробно" and a button labeled "Показать". In the center, there is a large empty rectangular box. On the right side, there are five buttons: "Заправка", "Передвижение", "Запустить", "Выключить", and "Заменить". Above the "Передвижение" button is a checkbox labeled "Высокая скорость".

Рисунок 5 – Оформление окна Windows Forms

4 Исходный код программы

Файл «Component.cs»:

```
using System;

public abstract class Component
{
    // Абстрактный класс Component, который реализует общие методы и события для всех компонентов автомобиля
    // Свойство Condition хранит текущее состояние компонента, по умолчанию "Новое"
    public string Condition { get; protected set; } = "Новое";

    // Метод ChangeCondition для изменения состояния компонента
    // Проверяет, что состояние может быть только "Новое" или "Сломано"
    // При изменении состояния на "Сломано" вызывает событие поломки компонента
    public void ChangeCondition(string condition)
    {
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        Condition = condition;

        // Если состояние компонента "Сломано", генерируем событие поломки
        if (Condition == "Сломано")
        {
            OnComponentBroken();
        }
    }

    // Защитный метод для вызова события поломки компонента
    // Срабатывает, если компонент сломался
    protected virtual void OnComponentBroken()
    {
        ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события ComponentBroken
    }

    // Событие поломки компонента, которое оповещает об изменении состояния на "Сломано"
    public event EventHandler ComponentBroken;
}
```


Файл «Form1.cs»:

```
using System;

// Класс Engine наследуется от Component и представляет двигатель автомобиля
// Класс управляет состоянием двигателя (работает/не работает) и его мощностью
public class Engine : Component
{
    // Свойство HorsePower хранит мощность двигателя в лошадиных силах
    public int HorsePower { get; set; }

    // Свойство IsRunning указывает, работает ли двигатель (true - работает, false - не работает)
    public bool IsRunning { get; private set; }

    // Метод Start запускает двигатель, если его состояние не "Сломано"
    // Если двигатель сломан, выбрасывается исключение
    public void Start()
    {
        if (Condition == "Сломано")
            throw new InvalidOperationException("Двигатель сломан, его нельзя запустить.");
        IsRunning = true;
    }

    // Метод Stop останавливает двигатель, если его состояние не "Сломано"
    // Если двигатель сломан, выбрасывается исключение
    public void Stop()
    {
        if (Condition == "Сломано")
            throw new InvalidOperationException("Двигатель сломан, его нельзя остановить.");
        IsRunning = false;
    }

    // Переопределение метода ChangeCondition для изменения состояния двигателя
    // Изменяет состояние и вызывает событие поломки, если двигатель стал "Сломан"
    public void ChangeCondition(string condition)
    {
        Condition = condition;
    }
}
```

5 Результаты работы программы с примерами

На рисунке 6 представлен результат работы программы с подклассом для колеса.

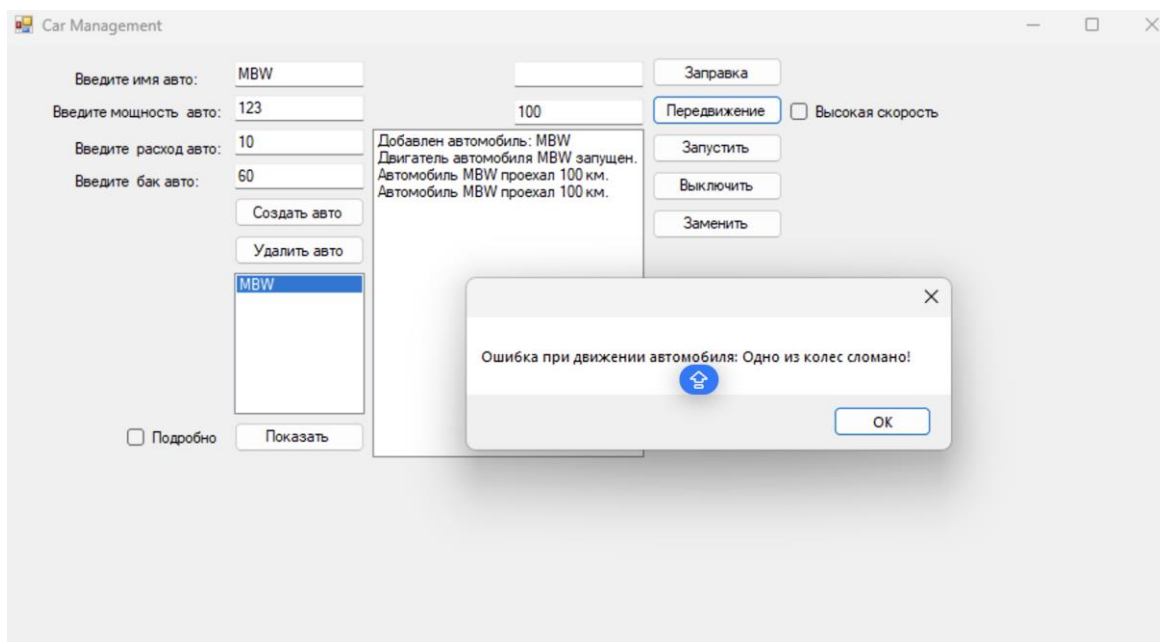


Рисунок 6 – Результат работы программы

На рисунке 7 представлен результат работы программы с подклассом для двигателя.

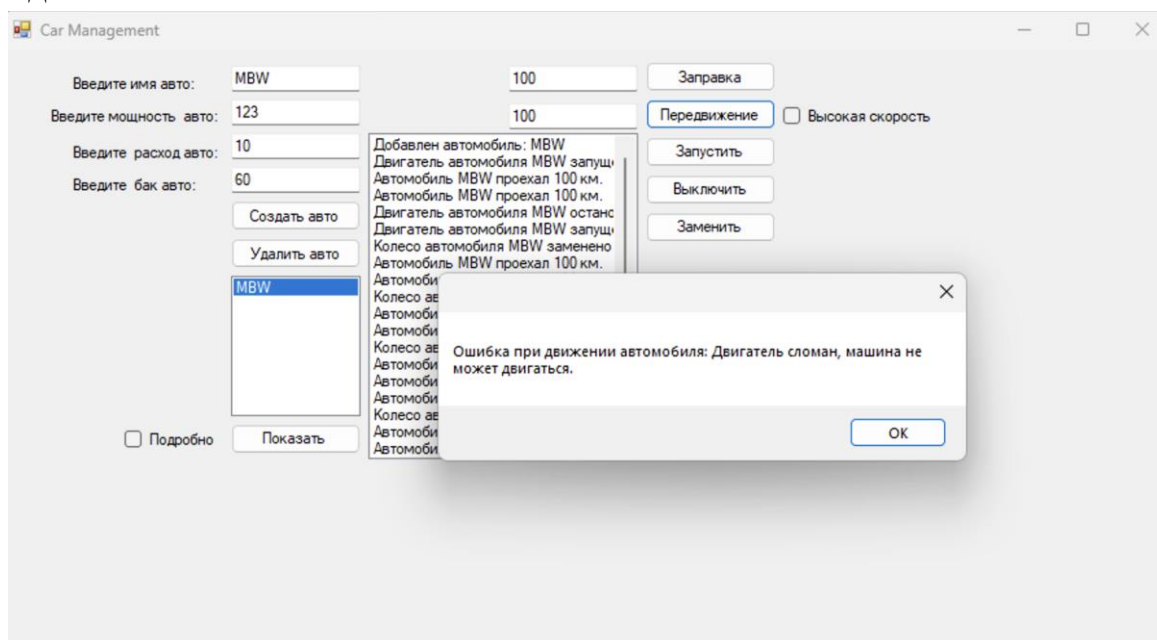


Рисунок 7 – Результат работы программы

6 Вывод

В ходе проделанной работы удалось достичь поставленных целей и изучить основы наследования на примере создания иерархий классов. Реализация проекта позволила на практике закрепить знания о построении классов, использовании базового класса и наследников, а также о механизмах восходящего и нисходящего преобразования типов. В процессе разработки была создана структура классов для моделирования автомобиля, включающая компоненты, такие как двигатель и колеса, а также их взаимодействие через события и методы. Это позволило продемонстрировать принципы инкапсуляции, полиморфизма и обработки исключений в случае поломки компонентов.

Разработанный проект также показал возможности использования нескольких конструкторов для создания объектов с различными параметрами, что повышает гибкость системы. В ходе тестирования приложения были продемонстрированы ключевые аспекты наследования, включая создание объектов подклассов через базовые классы и обратное преобразование типов для доступа к специфическим методам. Итогом работы стало не только успешное выполнение практического задания, но и углубление понимания принципов объектно-ориентированного программирования и их применения в реальных задачах.